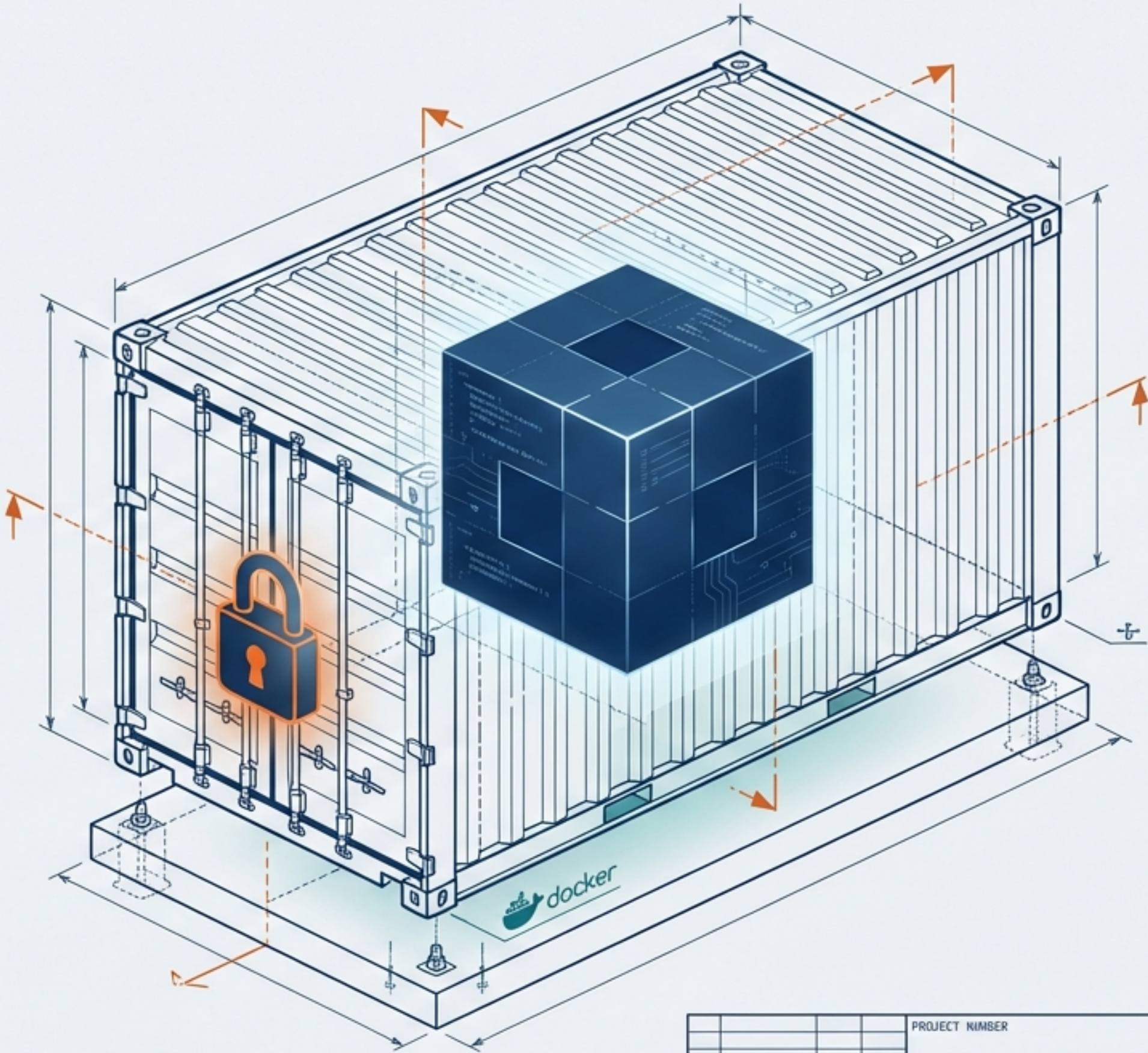


Containerising the Workspace

A Debrief on Refactoring sessionize-cli for Safe AI-Assisted Development.

VERSION	v0.2.3
DATE	26 January 2026
AUTHOR	Dinis Cruz
SUBJECT	Docker Environment Refactor



				PROJECT NUMBER
				PROJECT NUMBER
				REVISION DATE

Executive Summary: A Safe Harbour for Claude Code

THE CORE STATEMENT

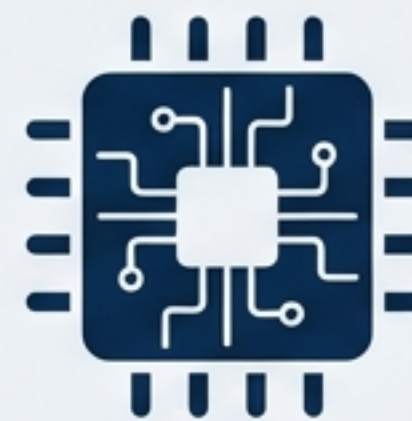
We refactored the sessionize-cli fork from a vulnerable local execution model to a robust Docker-based development environment.

THE DRIVER

The goal was to create a safe, isolated, and reproducible setup for running and improving the CLI tool using the AI agent, Claude Code.

KEY ACHIEVEMENT

The new architecture successfully mitigates risks associated with AI agents having direct host access while solving 'works on my machine' friction.



The Artifact and The Operator



The Tool: sessionize-cli

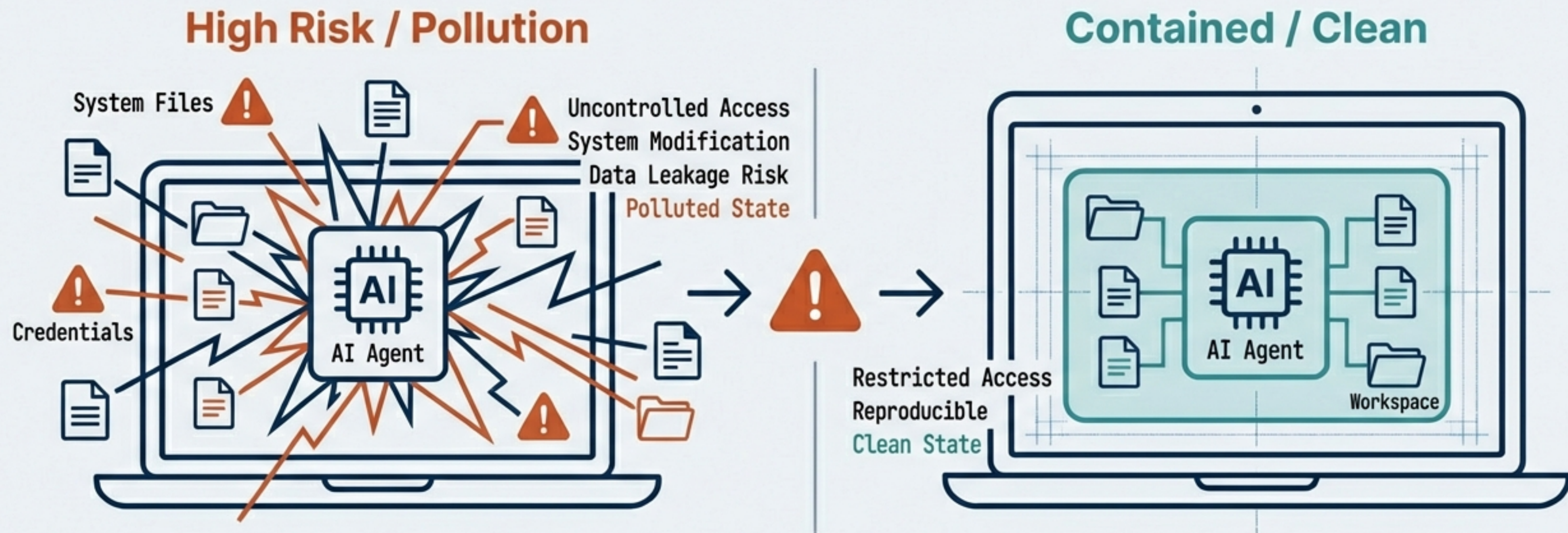
- **Origin:** Node.js CLI by csima
- **Function:** Rate sessions, search speakers, navigate submissions.
- **Dependency:** Browserbase (Cloud Automation)



The Operator: Claude Code

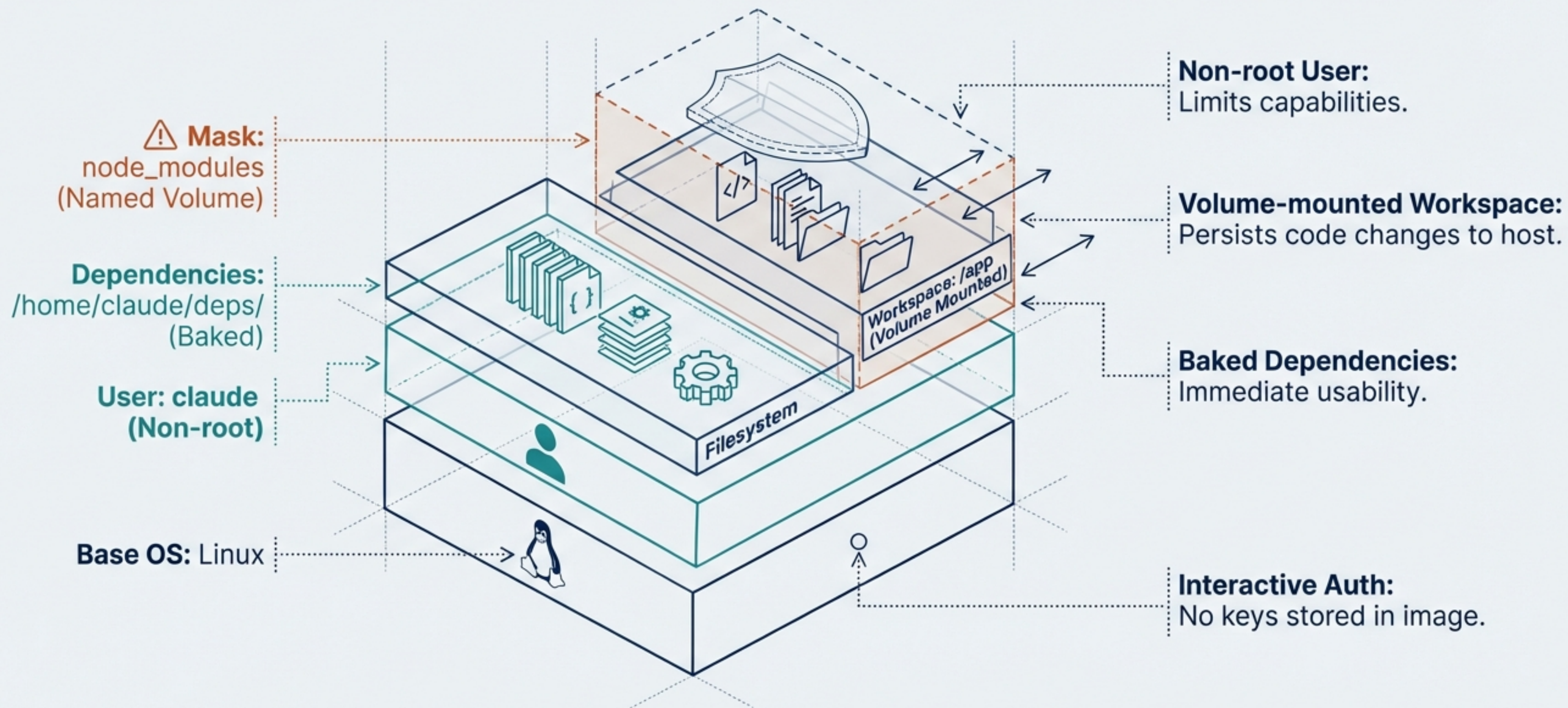
- **Role:** AI-Assisted Development Agent
- **Requirement:** Needs to run shell commands, install npm packages, edit files.
- **Constraint:** Must NOT have unrestricted host OS access.

Friction Points in Local Execution



1. **System Modification:** AI agents might unintentionally alter host system files or configurations.
2. **State Pollution:** Experiments leave residual files; 'Works on my machine' reproducibility issues.
3. **Security:** Granting an AI agent system-wide access endangers host credentials and keys.

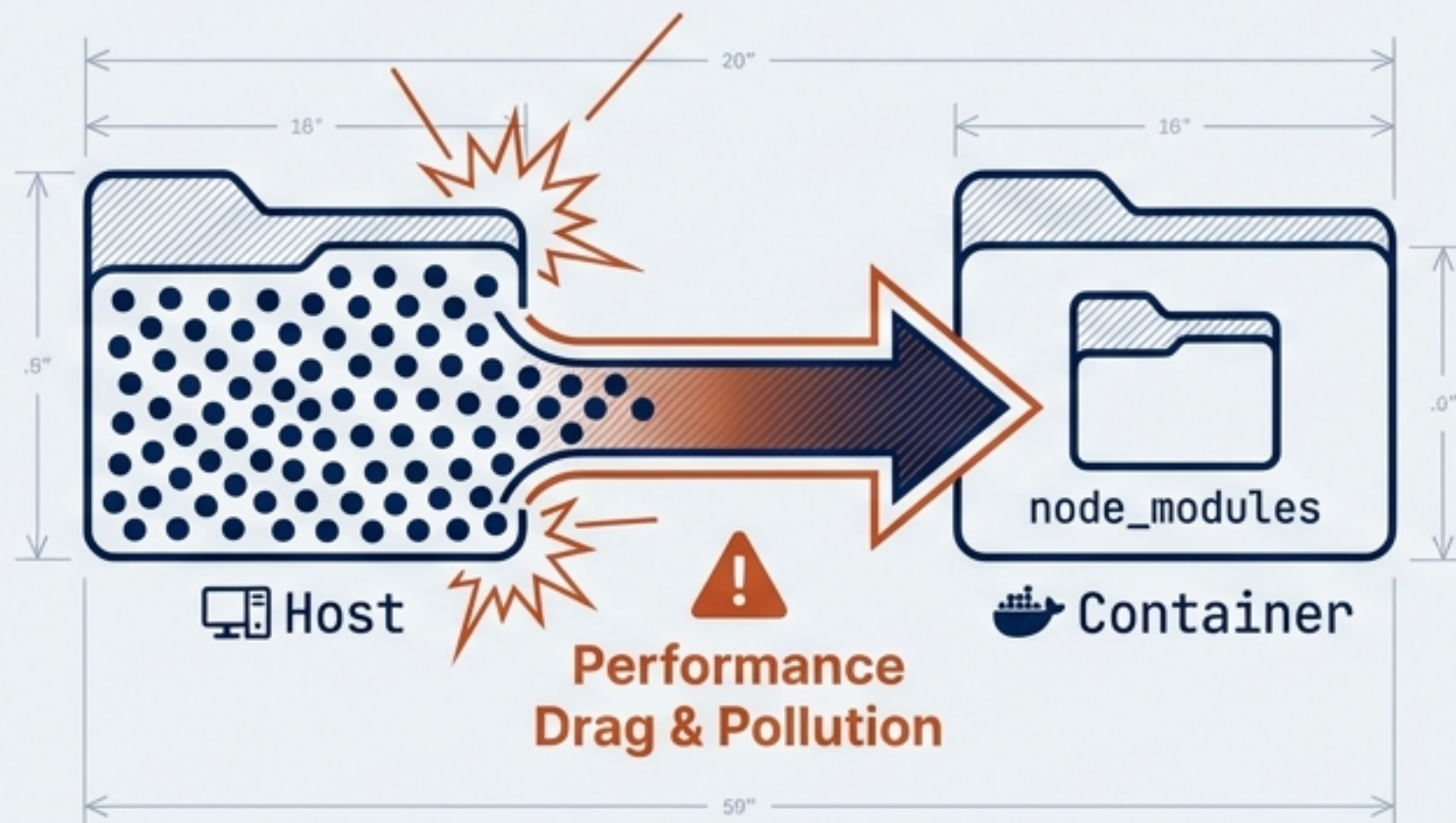
The Blueprint of the New Architecture



Engineering Challenge I: The node_modules Paradox

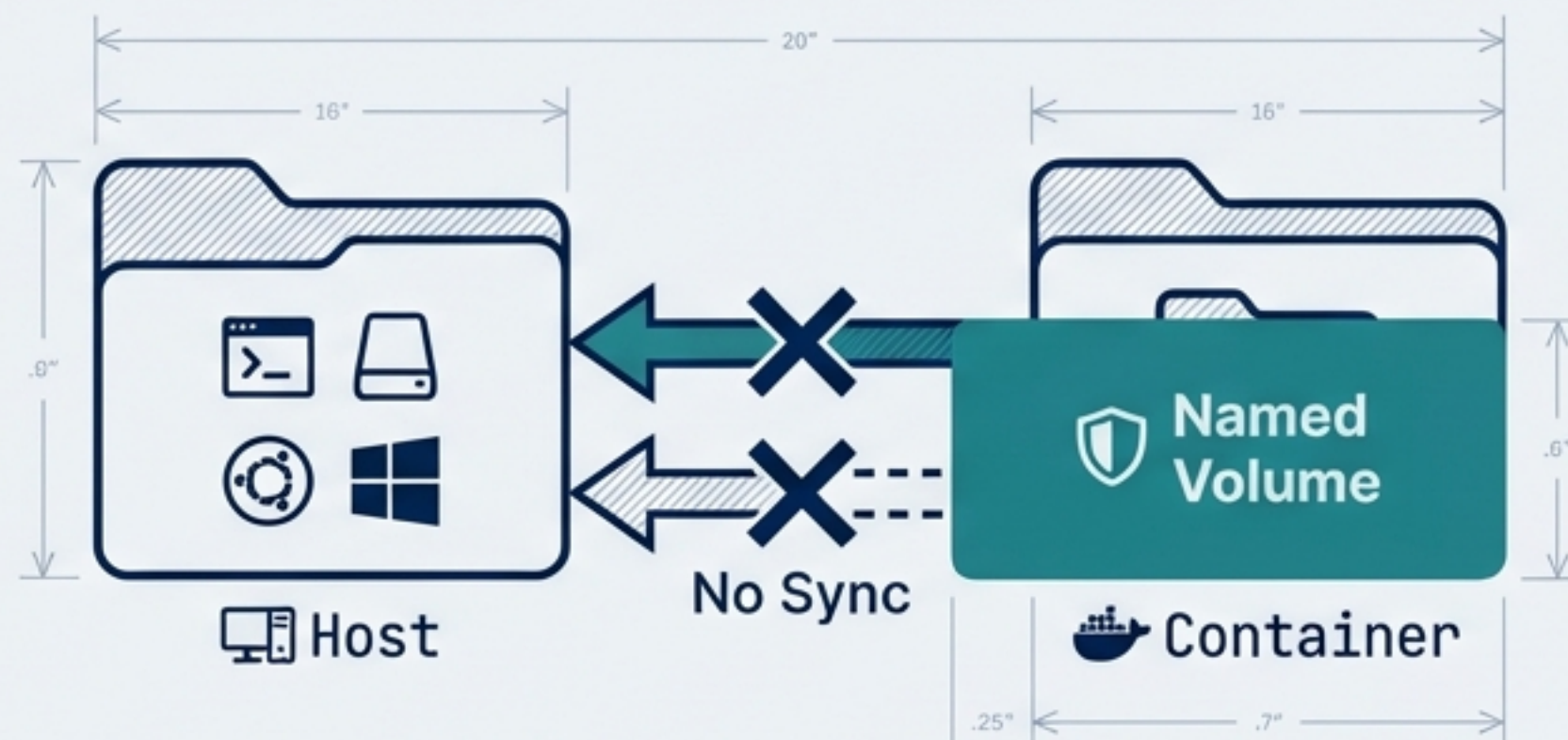
The Friction

Problem: Bind-mounting the workspace causes node_modules to sync back to the host OS.



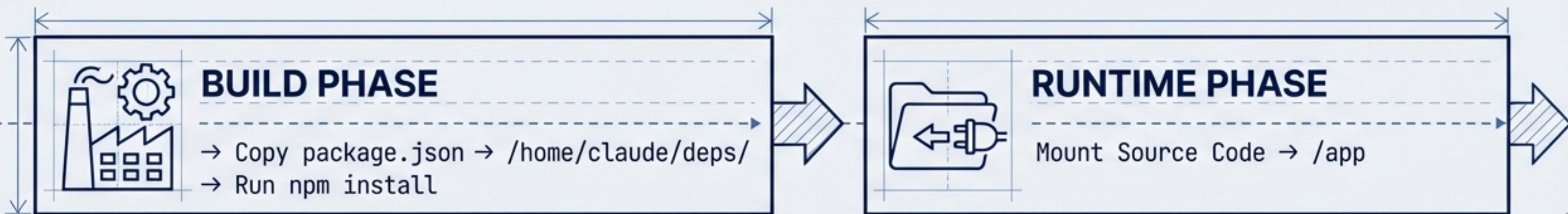
The Solution

Solution: Named Volumes as Masks.



```
1 volumes:
2   - ./workspace:/app # Bind Mount (Code)
3   - node_modules:/app/node_modules # Mask (Deps)
```


Engineering Challenge II: Build Time vs. Runtime

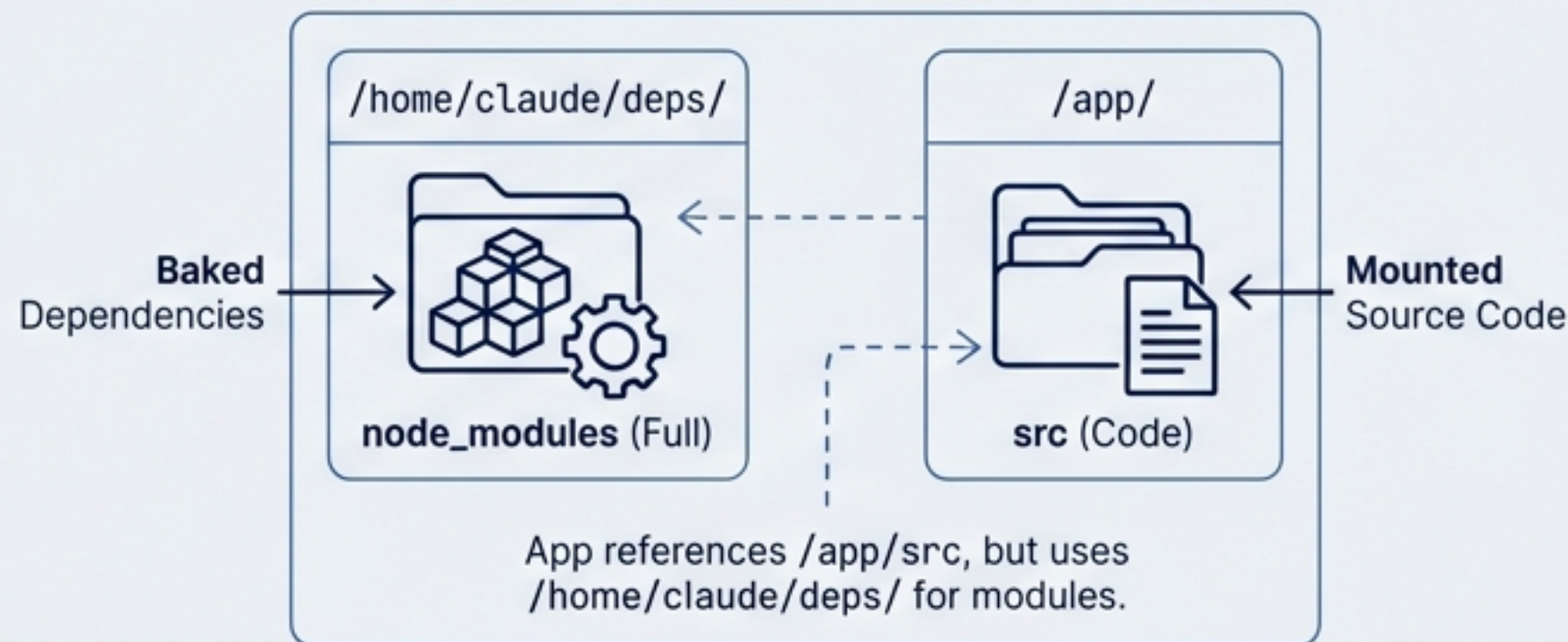


The Conflict

We need immediate usability (baked deps) but live editing (mounted code).

The Solution

Separation of Concerns. We install dependencies in a separate, non-mounted directory (/deps) during build. The app runs from there, referencing the mounted source code.



Engineering Challenge II: Permissions and Ordering

✗ Incorrect Ordering

```
# Permissions Fail  
RUN npm config set prefix ...  
USER claude  
# Result: npm writes to /root owned paths
```

⚠ Permission Denied on Global Install

✓ Correct Ordering

```
# Context Established First  
USER claude  
RUN npm config set prefix ...  
# Result: npm writes to /home/claude
```

✓ Successful Write Access

Lesson: In Dockerfiles, the order of operations strictly defines the permission context.

Engineering Challenge IV: Credential Containment

The Problem

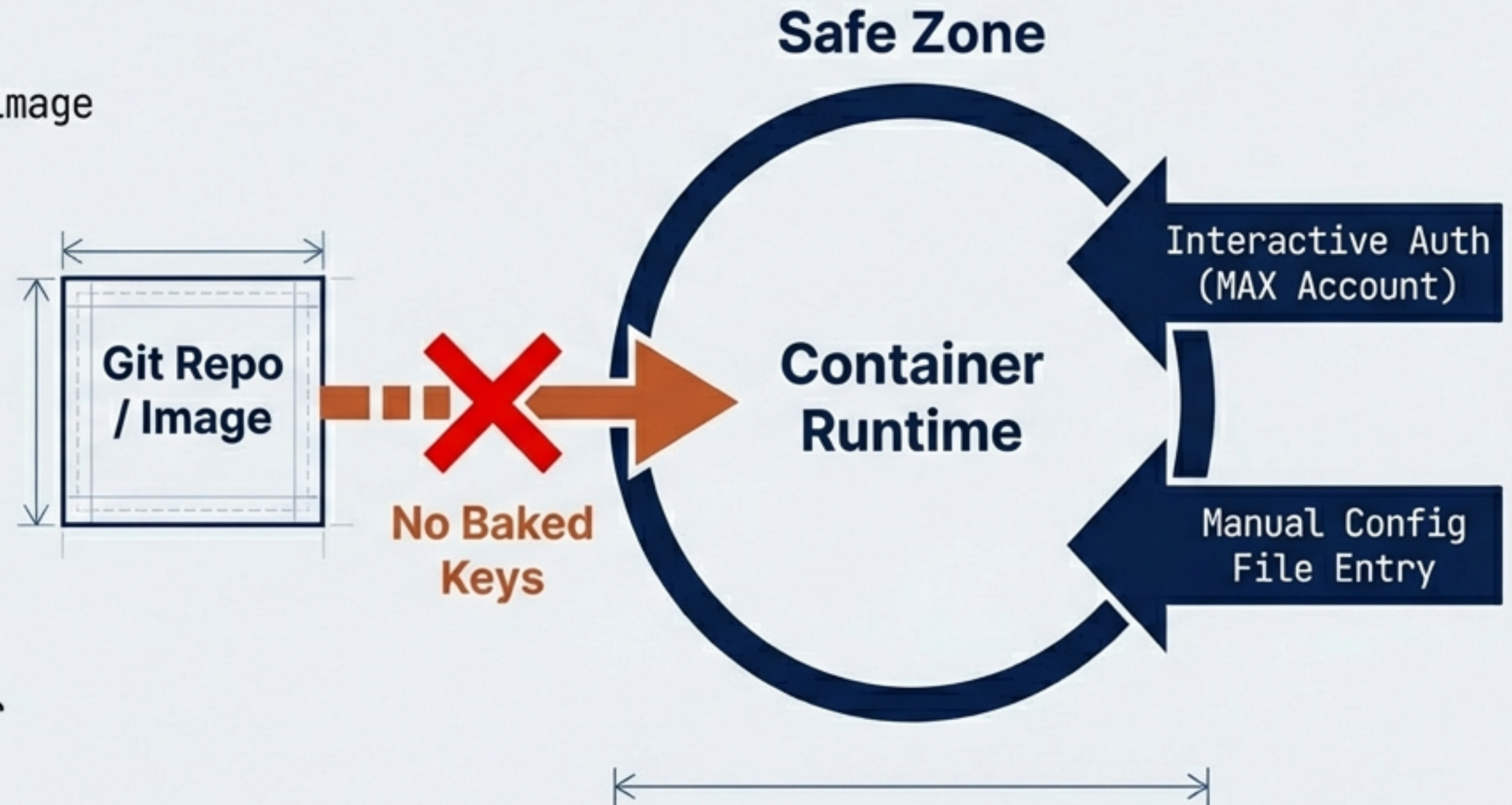
Baking API keys into the image is a security violation.

The Solution

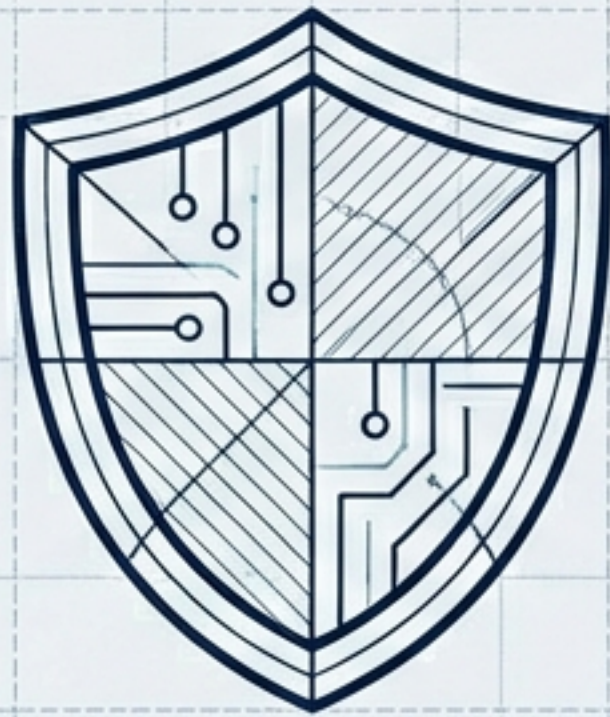
Interactive & Manual Configuration.

Rationale

Don't over-engineer credentials. Manual creation inside the container is simple, secure, and sufficient for development.



Three Pillars of the New Design



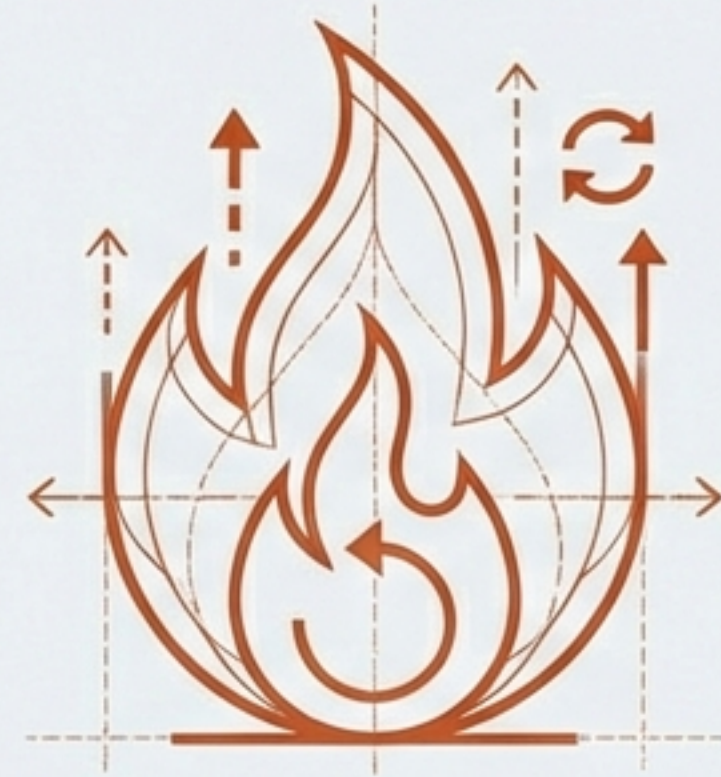
SAFETY

The host system is immutable to the agent. Docker prevents unintended changes to the engineer's OS.



REPRODUCIBILITY

Eliminates environment drift. Anyone can clone and run; the environment is mathematically identical.



EPHEMERALITY

The ability to wipe the slate clean.
'docker compose down -v' destroys all state instantly.

Operating Procedures



git clone ...
docker compose up



Authenticate Claude Code
via MAX account.

Manually add
Browserbase/Sessionize
credentials.



Edit files locally in
/workspace (Persisted).

Run code globally in
container (Contained).



Note: The container is git-unaware.
Version control happens safely on
the host machine.

Heuristics & Lessons Learned

Order Matters.

USER switching must precede configuration to ensure correct permission inheritance.



Masking is Powerful.

Named volumes effectively “hide” internal directories like `node_modules` from the host.



Bake & Separate.

Baking dependencies into a separate path enables immediate usability without polluting the workspace.



Reuse Patterns.

This refactor succeeded by building upon the proven foundation of the `claude-flow CRM` project.



Roadmap and Future Improvements



Credential Persistence

Implement volume mounting for credentials to avoid manual re-entry on restart.



Automation & CI/CD

Add Makefile for convenience commands and establish automated testing pipelines.



Multi-Architecture Support

Build images specifically targeting ARM64 for Apple Silicon performance.

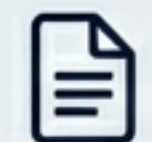
References & Credits



Original Repo: sessionize-cli by csima



Architecture Source: claude-flow CRM MVP (The Cyber Boardroom)



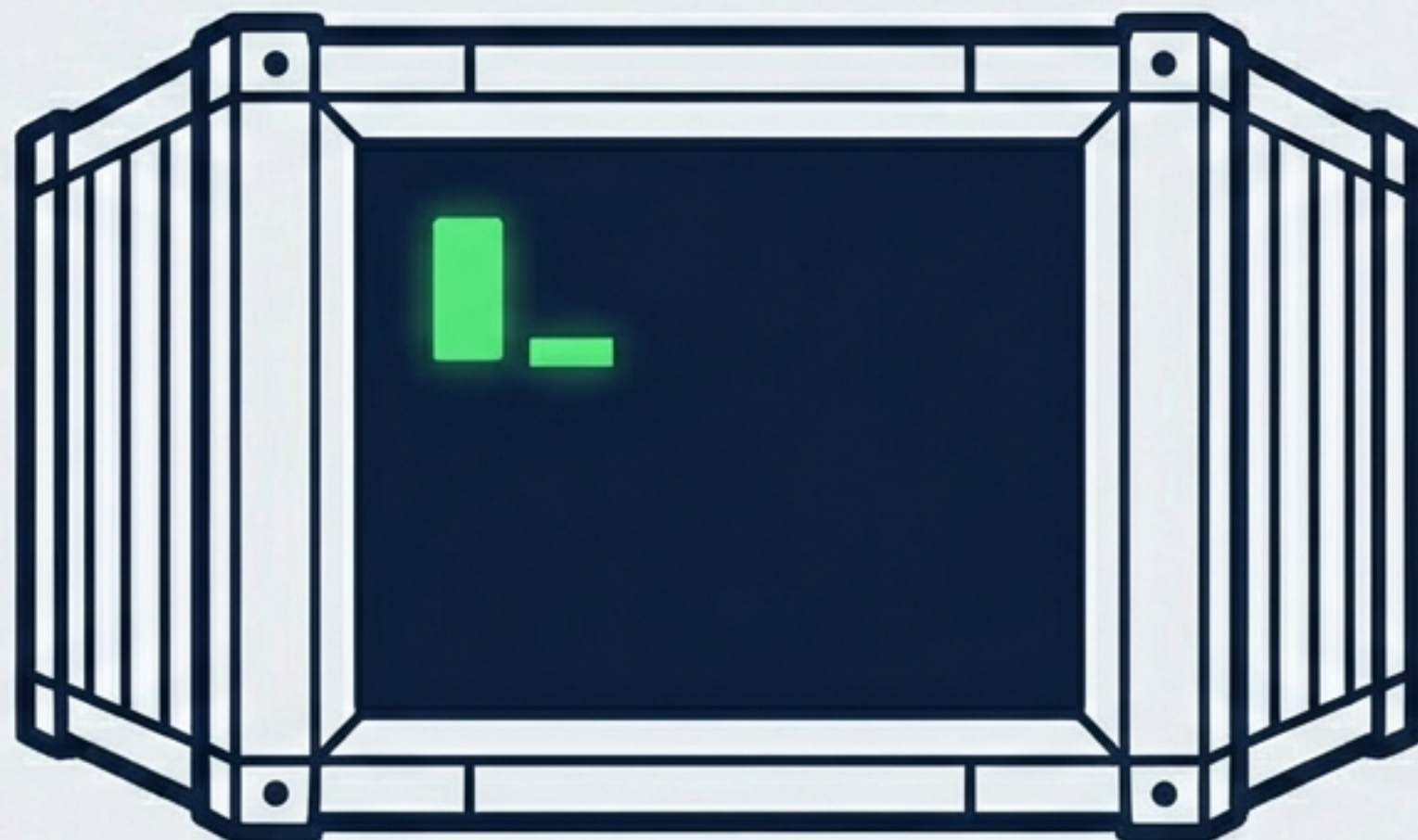
Design Doc: 006-docker-environment-design.md



Cloud Automation: Browserbase

v0.2.3 Debrief Complete

A Proven Foundation



We have successfully transformed a local CLI tool into
into a safe harbour for AI-driven development.